

Facial Biometric Verification System

banele mdluli

February 2026

1 Summary

Facial biometric verification is a technology that uses facial recognition algorithms to verify the identity of an individual based on their facial features. This technology has gained popularity in recent years due to its convenience and security benefits. The facial biometric verification system can be used in various applications, including access control, identity verification, and fraud prevention.

2 Introduction

Authentications and verifications are essential in various industries, including banking, healthcare, and government. Traditional methods of authentication, such as passwords and PINs, are vulnerable to hacking and identity theft. Facial biometric verification provides a more secure and convenient alternative to traditional methods. The system uses advanced algorithms to analyze facial features and compare them to a database of known faces to verify the identity of an individual.

3 Problem statement

The current methods are vulnerable to hackers, to improve the authentication checks, the project aims to develop a facial biometric system that can be integrated with other applications.

4 Objectives

The project objectives are targets that must be reached for the project to be considered a success. The objectives are:

- Develop a facial biometric system to authenticate users based on their ID number and photo.
- Develop a system that integrates with other applications for general use and enhanced validation process.

5 Requirements

5.1 Functional requirements

- Data quality checks
 - Data accuracy should be measured using the following methods:
 - * Compare two sources with the same data (if applicable)

- * Check if the cost or charge values are justified by the unit volume or quantity.
 - * Ingest the current day's data (must have a unique identifier) into an isolated storage and compare with tomorrow's version of the same data (use ID to trace the records).
- Data completeness must be measured in the following methods:
 - * Compare the latest record count and value (if applicable) to the previous day's record count and value. Acceptable thresholds can be established after reviewing outcomes
 - * Compare the latest record count and value to the previous month's same weekday record count and value.
- Data consistency should be measured by comparing the data field types of the current day with those of the previous day.
- Data uniqueness should be measured using the following methods:
 - * Check if there is a record ID with more than one record.
 - * Check if there is a record ID with different information from two or more records.
- Data timeliness should be measured using the following methods:
 - * Check if there are recent records ingested for the data source. Use the latest expected transaction date.
- Artificial intelligence agent
 - Create an agent that provides details about the data quality report. The agent should be able to provide an analysis that goes back in time (historical analysis).

5.2 Non functional requirements

- Performance-based requirements
 - The API should provide feedback within 5-7 secs. Or provide reasons for the delay.
 - For multilingual text, translate it into English while maintaining context.
 - A.I interactions with users must be logged as they are created while maintaining low latency.
 - An agent must admit when they do not know how to respond to a user's question and log the question in another table.
- Scalability requirements
 - The application should be packaged using containers and Kubernetes.

- During high peak traffic, the application should increase the number of running instances.
- During low trough traffic, the application should decrease the number of running instances.
- Reliability requirements
 - The application should automatically restart itself when it encounters an issue that leads to downtime.
 - The application should meet an 80.00% uptime (calculated daily).
- Maintainability requirements
 - The application must have a production, development and testing(pre-production) environment.
 - The application should use version control and a semantic versioning process (API not backwards-compatible feature: API backwards-compatible feature: API backwards-compatible bugfix).
 - the application should have automated tests using Azure DevOps or GitHub Actions.

5.3 Technical requirement

- A.I agent framework (**To be confirmed**)
- API dev framework
 - Fast API and documentation OpenAPI (Swagger)
- Frontend
 - React
 - Streamlit
- Environment packaging
 - docker
 - Kubernetes
- Deployment platform, code repo and CI/CD
 - Azure
 - Github
 - Github actions (switch to Azure DevOps later)

6 Design

The low-level design shows the structure and make-up of individual units. The LLD is a platform divided; units generated or developed on the same platform are part of the same LLD design.

TO BE UPDATED DURING DEVELOPMENT!!!!!!

7 High technical requirements

7.1 Fast API integrations

7.1.1 Postgres, Agent and Fast API

A PostgreSQL database should be used to store information analysed in Databricks. The information stored in the database will vary from data accuracy to validity checks.

- Create an Azure Postgres table to store data quality outcomes. Depending on the outcome size, one table can be used, or multiple tables (one check per table: if required).
- Create a Fast API parent endpoint that will interact with an agent or agents that will create a data quality check PDF report.
- Create a Decision matrix that should assist the agent in compiling the report to give a certain outcome.

7.1.2 Gradio, React and Fast API

- Create an agent that should interact with an external user and provide the desired outcome based on the information available on the PostgreSQL database. The agent should interact with the database using Fast API.
- Create a Gradio chat interface that is connected to the AI agent. Link the Gradio chat app to a React Interface using Fast API.
- Create a React platform that has the following:
 - Log in page with an option to sign up.
 - A page that shows operation indicators.
 - A page that shows KPI based performance indicators.
 - A page that allows users to interact with agents.
- Report data should be downloadable in various formats (csv, tsv, etc,...).

7.2 Databricks integrations

- Create a service principal in Azure and provide it Azure blob storage role.
- Assign the same service principal a role in the Azure Databricks workspace (contributor).
- Create tables and schemas that adhere to the data engineering architecture.
- Create an API that should be used to push data from the gold layer to an external PostgreSQL.

8 Discussion

9 Conclusion

10 Appendix

10.1 Code